Day 156 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 23, Day 3 reference

Watch: Content-Based Recommendations - Recommender Systems Part 1 (DataMListic) | Content-based Filtering (Mahesh Huddar)

`day156_content_based_filtering.py`, segment by segment, in the order `main()` uses things. The parts (load_years, film_features, content_similarity, fit_shared_profile, fit_profiles, content_predict, cold_split) are copied unchanged into `day156_exercise.py`; Day 154's parts are imported from `day154_exercise.py`. Code boxes are cut from the real script by program (inline comments removed). Purple **MATH** boxes come before code that implements a formula. A glossary is at the end.

MATH BOX LEGEND

Symbols: X = film features (films x 27) · T = what the user's mean (or the bias model) leaves · M = where ratings are known · w_0 = shared profile · W = per-user profiles · λ = penalty

Segment 0 — Setup

Imports, Day 154's part, settings

CODE

```
import contextlib
import io
import os
import sys
import time
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib.patches import FancyBboxPatch
if hasattr(sys.stdout, "reconfigure"):
    sys.stdout.reconfigure(encoding="utf-8")
HERE = os.path.dirname(os.path.abspath(__file__))
sys.path.insert(0, HERE)
with contextlib.redirect_stdout(io.StringIO()):
    import day154_exercise as d154
LU, LI = 5, 2
ITEM = dict(k=40, shrink=800)
LAMS = (0.3, 1, 3, 10, 30, 100, 300, 1000)
GLAMS = (0.01, 0.1, 1, 10, 100, 1000, 10000)
KS = (5, 10, 20, 40, 80, 160, 320)
DECADES = ("before 1930", "1930s", "1940s", "1950s", "1960s", "1970s", "1980s", "1990s")
```

- `with contextlib.redirect_stdout(io.StringIO()):` — Day 154's part prints its hand checks when imported; this sends them nowhere.
- `GLAMS = (0.01, ..., 10000)` — the shared profile's penalties span six powers of ten, so the choice is not at an edge.
- `DECADES = ("before 1930", ...)` — names for the 8 decade features, used when a profile is printed.

Segment 1 — The parts

load_years and film_features

CODE

```
def load_years(folder=d154.DATA):
    path = os.path.join(folder, "ml-100k", "u.item")
    rows = open(path, encoding="latin-1").read().strip().split("\n")
    return np.array([int(x.split("|")[2][-4:]) if x.split("|")[2] else -1 for x in rows])
def film_features(genres, years):
    dec = np.clip((years - 1920) // 10, 0, 7)
    D = np.eye(8)[dec] * (years > 0)[:, None]
    return np.c_[genres, D].astype(float)
```

- `def load_years(folder=d154.DATA):` — a default argument can be another module's constant: Day 154's data folder.
- `x.split("|")[2][-4:]` — the third field is the date, e.g. 01-Jan-1977; its last four characters are the year.
- `... if x.split("|")[2] else -1` — a conditional expression inside a list comprehension: -1 when the date is empty.
- `np.eye(8)[dec]` — rows of the 8 x 8 identity matrix: a one-hot row per film, picked by its decade number.
- `* (years > 0)[:, None]` — multiplying by a True/False column zeroes the decade flags of a film with no year.
- `np.c_[genres, D]` — joins the two tables side by side: 19 + 8 = 27 columns.

content_similarity

MATH

$$cos(a, b) = x_a \cdot x_b \,/\, (|x_a|\ |x_b|)$$

CODE

```
def content_similarity(X):
    Xn = X / np.maximum(np.linalg.norm(X, axis=1, keepdims=True), 1e-12)
    S = Xn @ Xn.T
    np.fill_diagonal(S, 0.0)
    return S
```

- `np.linalg.norm(X, axis=1, keepdims=True)` — each row's length, kept as a column so the division broadcasts across the row.
- `Xn @ Xn.T` — dot products of unit vectors: every pair's cosine in one matrix product.
- `np.fill_diagonal(S, 0.0)` — a film is never its own neighbour.

fit_shared_profile, fit_profiles, content_predict

MATH

$$w_u = w_0 + (X_u^T X_u + \lambda I)^{-1} X_u^T (t_u - X_u w_0)$$

CODE

```
def fit_shared_profile(X, T, M, lam):
    uu, ii = np.nonzero(M)
    A = X[ii]
    return np.linalg.solve(A.T @ A + lam * np.eye(X.shape[1]), A.T @ T[uu, ii])
def fit_profiles(X, T, M, lam, prior=None):
    w0 = np.zeros(X.shape[1]) if prior is None else prior
    W = np.tile(w0, (M.shape[0], 1))
    for a in range(M.shape[0]):
        j = np.flatnonzero(M[a]); A = X[j]
        W[a] += np.linalg.solve(A.T @ A + lam * np.eye(X.shape[1]), A.T @ (T[a, j] - A @ w0))
    return W
def content_predict(W, X, u, i):
    return (W[u] * X[i]).sum(1)
```

- `uu, ii = np.nonzero(M)` — the row and column of every True cell: one (user, film) pair per known rating.
- `A = X[ii]` — fancy indexing repeats a film's features once per rating of it.
- `np.tile(w0, (M.shape[0], 1))` — stacks the prior once per user: every profile starts at the shared one.
- `W[a] += np.linalg.solve(...)` — adds the ridge correction for what the prior misses; with few ratings it stays small.
- `(W[u] * X[i]).sum(1)` — one profile and one feature row per pair, multiplied and summed along the row.

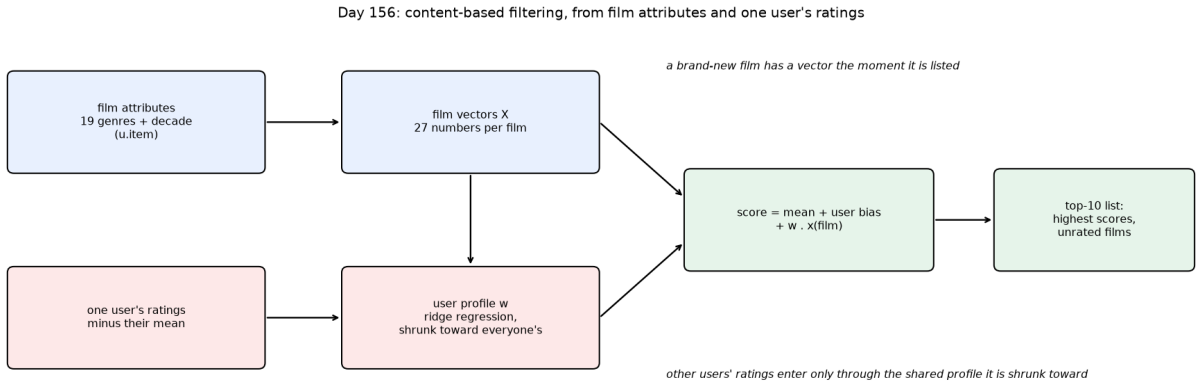
cold_split: holding out whole films

CODE

```
def cold_split(i, ni, frac, seed, exclude=None):
    pool = np.arange(ni) if exclude is None else np.flatnonzero(~exclude)
    rng = np.random.RandomState(seed); cold = np.zeros(ni, bool)
    cold[rng.permutation(pool)[:int(round(frac * ni))]] = True
    return cold, np.flatnonzero(cold[i])
```

- `np.flatnonzero(~exclude)` — ~ flips a True/False array: the films not already held out.
- `int(round(frac * ni))` — the number of films to hold out, a fraction of all films.
- `np.flatnonzero(cold[i])` — cold[i] is True for each rating whose film is held out; their positions are returned.

SYSTEM DIAGRAM



Content-based filtering.

Segment 2 — The experiments

Helpers: the user-only model, picking, top-10

CODE

```
def user_only(R, M):
    mu, bu, _ = d154.fit_biases(R, M, LU, 1e12)
    return mu, bu
def pick(errors, grid):
    return grid[int(np.argmin(errors))]
def top10(Q, M, liked):
    Q = np.where(M, -np.inf, Q); who = np.flatnonzero(liked.sum(1) > 0)
    top = np.argsort(-Q, axis=1, kind="stable")[:, :10]
    return np.take_along_axis(liked, top, 1)[who].sum(1).mean() / 10, top[who]
```

- `d154.fit_biases(R, M, LU, 1e12)` — a huge film penalty keeps every film bias at 0: the model a brand-new film can use.
- `mu, bu, _ = ...` — the underscore takes a returned value that is not needed.
- `grid[int(np.argmin(errors))]` — the setting whose validation error is lowest; argmin returns the first on a tie.

demo_content: does content agree with ratings?

CODE

```
nb = np.argsort(-Sr[np.ix_(ok, ok)], axis=1)[:, :10]
near = S[ok[:, None], ok[nb]].ravel()
rng = np.random.RandomState(0); rnd = rng.randint(0, len(ok), nb.shape)
far = S[ok[:, None], ok[rnd]].ravel()
```

- `Sr[np.ix_(ok, ok)]` — the rating similarities among the well-rated films only.
- `S[ok[:, None], ok[nb]]` — a column of row indices with a table of column indices: each film's content cosine with its own 10 neighbours.
- `rng.randint(0, len(ok), nb.shape)` — random positions with the same shape as the neighbour table: the control.
- `.ravel()` — flattens the table into one long list of cosines.

demo_profiles: the grid and the bins

CODE

```
for lam in LAMS:
    W = fit_profiles(X, T, Mf, lam, prior=wg)
    Ec.append(score(mu + bu[u[va]] + content_predict(W, X, u[va], i[va]), r[va]))
    W = fit_profiles(X, Tb, Mf, lam, prior=wgb)
    Eb.append(score(B[u[va], i[va]] + content_predict(W, X, u[va], i[va]), r[va]))
for lo, hi, nm in zip(edges[:-1], edges[1:], names):
    q = (cnt >= lo) & (cnt < hi)
```

- `for lam in LAMS:` — both models are refitted for every penalty and scored on validation.
- `zip(edges[:-1], edges[1:], names)` — pairs each bin's lower edge with its upper edge and its label.
- `(cnt >= lo) & (cnt < hi)` — & combines two True/False arrays element by element; the brackets are needed.

demo_cold: validation on new films

CODE

```
cold, te = cold_split(i, ni, 0.2, seed=2)
cval, va = cold_split(i, ni, 0.1, seed=3, exclude=cold)
fi = np.flatnonzero(~cold[i] & ~cval[i])
S = content_similarity(X); Ek = []
for k in KS:
    c, _ = d154.predict_item_knn(T, M, S, u[va], i[va], k)
    Ek.append(score(mu + bu[u[va]] + c, r[va]))
```

- `exclude=cold` — the validation films are drawn from the films not already in test.
- `~cold[i] & ~cval[i]` — training keeps only ratings of films in neither held-out set.
- `d154.predict_item_knn(T, M, S, ...)` — Day 154's item-based predictor, unchanged, given the content similarity S.

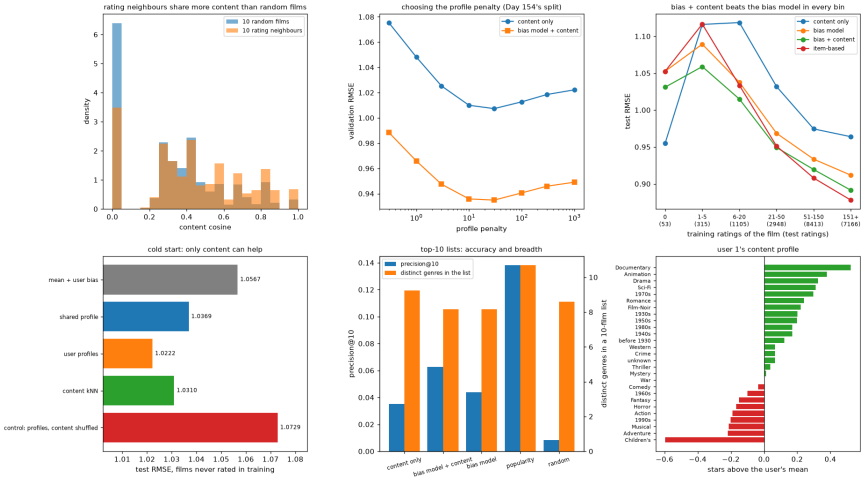
demo_topn and explain_list: lists and their reasons

CODE

```
dg = np.mean([np.count_nonzero(genres[t].sum(0)) for t in top])
s = np.where(rated[a], -np.inf, Wc[a] @ X.T); top = np.argsort(-s, kind="stable")[:5]
for j in top:
    f = np.flatnonzero(X[j]); parts = sorted(((Wc[a, q], names[q]) for q in f), reverse=True)
    print(f"  {titles[j][:38]:<38} {s[j]:+.2f} = " + " ".join(f"{w:+.2f} ({n})" for w, n in
        parts))
```

- `genres[t].sum(0)` — how many of the 10 films have each genre; count_nonzero counts the genres present.
- `Wc[a] @ X.T` — user 1's profile against every film: one score per film.
- `sorted(((Wc[a, q], names[q]) for q in f), reverse=True)` — pairs (weight, name), largest weight first; tuples sort by their first item.
- `titles[j][:38]:<38` — the first 38 characters, padded to 38 so the columns line up.

REAL GRAPH



Real graph from the script.

Quick Syntax Glossary — new or reinforced constructs

- `np.eye(n)[idx]` — one-hot rows.
- `np.c_[A, B]` — join tables side by side.
- `np.linalg.norm(X, axis=1, keepdims=True)` — row lengths as a column.
- `np.nonzero(M)` — row and column positions of True cells.
- `np.tile(v, (n, 1))` — repeat a row n times.
- `~mask` — flip True and False.
- `a & b` — element-wise and of two masks.
- `A[rows[:, None], cols]` — pick a different set of columns for each row.
- `x.ravel()` — flatten to one dimension.
- `sorted(pairs, reverse=True)` — largest first.

End of Day 156 syntax reference. For the experiments and what they showed, see the Day 156 complete guide.